

THE EASY VERSION

Your talk, explained simply

Break My API: Can AI Find the Bugs I Forgot to Test?

What to tell tomorrow · the background · why this was built · how the code works · how to run it. One idea per page, with a simple example on each.

STEP 1

The whole talk in 5 lines

If you remember only this page, you can give the talk.

1

The idea. Most people ask AI to *write* code. I asked AI to *break* my code.

2

What I built. A small online ticket shop with 10 mistakes (bugs) hidden in it on purpose.

3

What the AI did. It wrote 8 tests to try to break the shop. A small program ran each test.

4

What happened. 3 tests found real bugs. 2 were the AI being wrong. It missed the biggest bug, which I found.

5

The lesson. AI makes guesses. Running them gives proof. A person decides what's really a bug.

THE EVENT THEME: "BEYOND THE PROMPT"

Asking the AI (the prompt) was the easy part. The checking I built around it is what made the AI useful. **That is "beyond the prompt".**

SIMPLE EXAMPLE

It's like a new junior tester on your team: fast, full of ideas, finds some real problems, sometimes wrong. You still check their work.

STEP 2 · BACKGROUND

What is an API?

An API is how one program asks another program for something.



You order through the waiter. You never go into the kitchen.

A request and an answer

SIMPLE EXAMPLE

You ask: "Buy 2 tickets for me."

The API answers: "201 Created. You paid 4000." The number **201** means "it worked".

The answer codes

Code	Meaning	Example
2xx	It worked	Your order was placed
4xx	You asked for something wrong	"You can't buy -5 tickets"
5xx	The server broke	A crash

Key idea: if you send something wrong, a good API answers **4xx**. If it answers **2xx** or **5xx**, that's usually a bug.

STEP 3 · BACKGROUND

What is testing, and why do we miss bugs?

A test is: do something, and check you got the answer you expected.

SIMPLE EXAMPLE OF A TEST

Buy 2 tickets → expect **201** and **4000 charged**. If that's what comes back, the test

PASSES.

The "happy path"

Developers usually test the normal way people use a feature: buy 2 tickets, log in with the right password. That's called the **happy path**.

What real people and attackers send

- Minus five tickets (**-5**)
- The same "use once" coupon, twice
- A fake login key they made up
- Twenty people buying the last 5 seats at the same second

WHY WE MISS THESE

When I write the code, I already "know" quantity will be positive. So I never think to test **-5**. **The person who writes the code shares its blind spots.**

SO THE QUESTION OF MY TALK IS

Can AI think of the tests I forgot?

STEP 4

Why this project was written

To answer one question with real evidence, not opinions.

Why a fake shop?

I needed an app where I **already know** all the bugs. Then I can check exactly what the AI found and what it missed. It's small, safe, and resets in a second.

Why plant bugs on purpose?

Like a teacher hiding 10 mistakes in an essay to see how many a student finds. The answers are known, so the score is fair.

Why a local AI on my laptop?

Reason	What it means
Free	No paid account needed
Offline	Works even if the venue Wi-Fi fails
Private	My code never leaves my laptop
Made for code	The model is called qwen2.5-coder:7b

THE TRADE-OFF

It's a small model, so it's slower (about 90 seconds) and makes more mistakes than big online AIs. That's fine: it shows why checking matters.

STEP 5 · THE CODE

How the code is organised

Four folders, like four people with four jobs.

**app/ · the shop**

The ticket shop itself, with the 10 hidden bugs.

main.py, db.py

**ai/ · the attacker**

The instructions for the AI and the tests it wrote.

prompts.md, generate.py, run_*.json

**tests/ · the referee**

Runs every AI test against the shop and says PASS or FAIL.

runner.py

**demo/ · special shows**

Two bugs the AI's tests can't show: the money loop and the race.

money_loop.py, race.py

Written in

Python. The shop uses **FastAPI** (to make the API) and **SQLite** (a small database in one file). The AI runs with **Ollama**.

SIMPLE WAY TO SAY IT

"The shop has bugs. The AI attacks it. The referee checks each attack. The demos show what the referee can't."

STEP 6 · THE CODE

How the shop code works: one example

What happens when someone buys tickets, line by line.

```
# 1. Is the quantity too big?
if body.quantity >= 10:
    raise HTTPException(400, "max 10 tickets per order")

# 2. Work out the price
total = price * quantity * (100 - discount) / 100

# 3. Take the money, give 10% back as credits
credits = credits - total + price * quantity * 10%
```

In plain words

- 1 It checks the quantity isn't too **big**. But it never checks it isn't too **small** (like **-5**).
- 2 It calculates the price, minus any coupon discount.
- 3 It takes the money and gives 10% back. But the 10% uses the **full price**, even if you paid nothing with a coupon.

SIMPLE EXAMPLE: THE -5 BUG

Buy **-5** tickets at 2000 each:

Price = $2000 \times -5 = -10,000$. The shop "charges" minus 10,000, which means it **gives you** 10,000. The API answers **201**: it worked. **That's a bug.**

STEP 7 · THE AI

How the AI part works

Three steps: ask, get tests back, check them.

1

Ask (the prompt). I tell the AI: "You are an attacker. Here is my shop. Find ways to break it. For each test, say what a *correct* shop should answer."

2

Get tests back. The AI answers in a fixed format called JSON, so a program can read it.

3

Check the format. If a test is broken or missing something, it's rejected automatically. (1 of 9 was rejected.)

SIMPLE EXAMPLE: ONE TEST THE AI WROTE

```
{ "endpoint": "/orders/1",      ← look at order number 1
  "auth": "bob",               ← logged in as Bob
  "expect_status": [403],      ← correct answer: "not allowed"
  "reason": "Bob may be able to read Alice's order" }
```

In plain words: "Order 1 belongs to Alice. If Bob asks for it, a correct shop should say 403: not allowed."

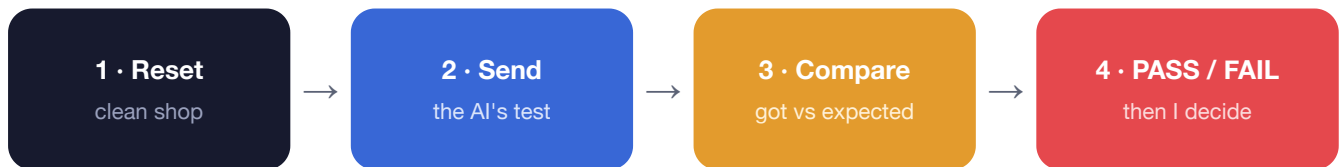
THE MOST IMPORTANT INSTRUCTION

"Tell me what a **correct** shop should answer, not what this shop answers."
Otherwise the AI just copies the current behaviour, and every test passes.

STEP 8 · THE CHECKING

How the referee (runner) works

It's simple on purpose: send the test, compare the answer.



SIMPLE EXAMPLE: THE BOB TEST

Expected: 403 (not allowed). **Got:** 200 (here's Alice's order).

They don't match → **FAIL**. I check it: yes, Bob really can read Alice's order. **Real bug.**

What PASS and FAIL really mean

Result	Means	Does NOT mean
PASS	This guess didn't find a problem	"There are no bugs"
FAIL	Maybe a bug: a person must check	"Definitely a bug"

That's why a human is the last step. The AI can be wrong about what "correct" is.

STEP 9 · RESULTS

What happened

The AI wrote 8 tests. 5 came back FAIL. Here is each one, in plain words.

✅ 3 real bugs (the AI was right)

- **Broken login key** → **the server crashed** (500) instead of saying "invalid key" (401).
- **Fake login key** → **admin access**. The key isn't locked; anyone can make one.
- **Bob could read Alice's order** just by changing the order number.

❌ 2 false alarms (the AI was wrong)

- It said a correct shop should **crash** (500) for a hacking attempt. No: crashing is never correct.
- It thought the **EARLYBIRD** coupon could be used only once. The rules say it's unlimited.

🔍 What only the human found

- **The money loop**: free coupon + 10% credits from the full price = you get richer every time you "buy". 5,000 became 14,000 while paying 0.
- **The race**: 20 people bought 5 seats at the same time. All 20 succeeded.

IN ONE LINE

When the AI said "bug", it was right **3 times out of 5 (60%)**, in about 90 seconds. Useful and fast, but it still needs checking.

STEP 10 · HANDS ON

How to run it yourself

You need two terminal windows. Window 1 runs the shop. Window 2 runs the tests.

Window 1: start the shop (leave it running)

```
cd ~/Downloads/break-my-api  
.venv/bin/uvicorn app.main:app --port 8000
```

You'll see "Uvicorn running on http://127.0.0.1:8000". Open that address + `/docs` in a browser to click around the shop.

Window 2: the demos

1

Run the AI's tests

```
.venv/bin/python -m tests.runner --cases ai/run_prompt2.json
```

You'll see: **3 matched expectation, 5 did not.**

2

The money loop

```
.venv/bin/python demo/money_loop.py
```

You'll see the balance go from 5,000 to 14,000.

3

The race

```
.venv/bin/python demo/race.py
```

You'll see: 20 accepted, oversold by 15.

4

Clean up the shop's data

```
curl -s -X POST localhost:8000/_reset
```

STEP 11 · TOMORROW

What to say tomorrow

Read this out loud three times tonight. That's enough.

OPENING (1 MINUTE)

"Most of us use AI to write code. I wanted to know: can it break code?"

So I built a small ticket shop with ten bugs hidden on purpose. Then I asked a local AI to attack it. It wrote eight tests; today we run them live.

Three found real bugs my own tests had missed. Two were the AI being confidently wrong. And the most expensive bug, a trick that creates money from nothing, the AI never found. I did, because I know how the business works.

So: AI generates guesses. Running them gives evidence. Humans decide what's really a bug. Let me show you."

If someone asks...

Question	Easy answer
Did AI actually help?	Yes: 3 real bugs in 90 seconds that my tests missed. I just check its work.
Which AI did you use?	qwen2.5-coder, running on my laptop: free, private, offline.
Is this a unit test?	No, it tests the whole running app from outside, like a real user.
Something you don't know	"Good question. I haven't tested that; here's how I would."

YOU'VE GOT THIS

You built this.

You ran it.

You found the bug the AI missed.

Nobody in the room knows this project better than you. You don't need to know everything; you need to tell the story you lived.

AI makes guesses. Running them gives proof. People decide.